

PAR Laboratory Assignment

Lab basics: Experimental setup

Mario Acosta, Eduard Ayguadé, Rosa M. Badia, Jesús Labarta, Pau Ferrer,
Daniel Jiménez-González, Pablo Vizcaino, and Gladys Utrera

Spring 2025–2026



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Departament d'Arquitectura de Computadors

Contents

1	Experimental setup	7
1.1	Accessing the boada cluster	7
1.2	Node architecture and memory	8
1.3	Execution modes: interactive vs. queued	9
1.4	Serial compilation and execution	10
1.5	Compilation and execution of OpenMP programs	11
1.6	Strong vs. weak scalability	12
1.7	Discussion at the laboratory	13
A	Accessing the boada cluster	15

Objectives

At the end of this laboratory you should be able to:

1. Discover the architecture of a node in our cluster environment and understand its main parameters.
2. Compile and execute sequential and OpenMP parallel codes in our cluster environment.

Chapter 1

Experimental setup

The objective of this laboratory session is to familiarise yourself with the hardware and software environments that you will be using during this semester to carry out all the laboratory assignments in PAR.

1.1 Accessing the boada cluster

You can access boada, a multiprocessor server located at the Departament d'Arquitectura de Computadors (DAC), from a local Linux terminal. To access the boada server from a computer outside the UPC you will need to use UPC's VPN connection. You need to establish a connection using the secure shell (SSH) command:

```
ssh -X par????@boada.ac.upc.edu
```

being `????` the user number assigned to you. The option `-X` is necessary to forward the X11 commands opening remote graphical windows in your local desktop.

We recommend you to change the password of your account by typing:

```
ssh -t par????@boada.ac.upc.edu passwd
```

After entering the old password correctly twice, you will be asked to enter a new password also twice. Annex A explains how to access boada from your personal computer.

Once you log in to boada, you will find yourself in any of the interactive nodes, i.e. `boada-11` to `boada-13`, where you can execute *interactive* jobs and from where you can submit *execution* jobs to other nodes of the machine. In fact, boada is composed of several nodes equipped with different processor generations as shown in Table 1.1:

Table 1.1: Nodes of the boada cluster.

Node name	Processor generation	Interactive	Partition
<code>boada-[6-8]</code>	2× Intel Xeon E5-2609 v4	No	execution2
<code>boada-9</code>	Intel Xeon E5-1620 v4 + Nvidia K40c	No	CUDA9
<code>boada-10</code>	Intel Xeon Silver 4314 + 4× Nvidia GeForce RTX 3080	No	CUDA
<code>boada-[11-13]</code>	2× Intel Xeon Silver 4210R	Yes	interactive
<code>boada-15</code>	Intel Xeon Silver 4210R + ASUS AI CRL-G116U-P3DF	No	iacard
<code>boada-[18-20]</code>	2× Intel Xeon Silver 4314	No	execution*

In this course you are going to use nodes `boada-11` to `boada-13` interactively and nodes `boada-18` to `boada-20` through the execution partition, as explained in subsequent sections.

All the nodes have access to a shared network attached storage (NAS) disk. You can access it through

```
cd /scratch/nas/1/par????
```

which corresponds to your *home* directory (also referred to as `~/`).

All the necessary files to carry out each laboratory assignment will be posted in
`/scratch/nas/1/par0/sessions`

For today's session, copy the file `lab1.tar.gz` from that location to your home directory:

```
cp /scratch/nas/1/par0/sessions/lab1.tar.gz ~/
```

and, from your home directory, uncompress it with this command line:

```
tar -zxvf lab1.tar.gz
```

Finally, in order to set up all the environment variables, you have to process the `environment.bash` file now available in your home directory with:

```
source ~/environment.bash
```

Note that you must load this environment *every time* you log in to boada. Alternatively, you can add this command to your `.bashrc` so that the environment will be automatically load every time you access the cluster. You can achieve this by executing the following command from your home directory:

```
echo 'source ~/environment.bash' >> .bashrc
```

In case you need to transfer files from boada to your local machine (laptop or desktop in the laboratory room), or vice versa, you have to use the secure copy `scp` command. For example, if you type the following command in your local machine:

```
scp -O par????@boada.ac.upc.edu:lab1/pi/pi_seq.c .
```

you will copy the source file `pi_seq.c` located in `lab1/pi` at your home directory in boada to the current directory in your local machine, represented with the final dot (`.`). Note that the command above maintains the original name (`pi_seq.c`) for the copied file. The option `-O` is included for compatibility reasons.

1.2 Node architecture and memory

We propose you to investigate the architecture of the available nodes in boada. Go to `lab1/arch` directory and execute the following command:

```
sbatch submit-arch.sh
```

The `sbatch` command above enqueues the `submit-arch.sh` script that executes the `lscpu`, `lstopo` commands, and `cat` of a system file, to retrieve information about the hardware in one of the execution nodes (i.e. from `boada-18` to `boada-20`, thereby generating four files:

- `lscpu-boada-???.txt`,
- `lstopo-boada-???.txt`,
- `map-boada-???.fig`,
- `cache_line_size-boada-???.txt`,

where `??` may be 18, 19, or 20, referring to one of the execution nodes of boada.

You can execute the command:

```
xfig map-boada-???.fig
```

to visualise the output file and export to a different format (e.g. pdf or jpg) using **File** → **Export** in order to keep this information.

In boada you can execute the command (where `???` refers to a name):

```
xpdf ????.pdf
```


to open pdf files and

```
display ????.jpg
```

to visualise graphics files. You can also type:

```
fig2dev -L pdf ????.fig ????.pdf
```

to convert from .fig to .pdf formats; you can look for alternative output graphic languages by typing:

```
man fig2dev
```

Alternatively, remember that you can always take screenshots by pressing the dedicated **Prts** keyboard button.

The four generated files will help you to figure out:

- the number of sockets, cores per socket, and threads per core in a specific node;
- the amount of main memory in a specific node, and each NUMA node;
- the cache memory hierarchy (i.e. L1, L2, and L3), private or shared to each core/socket;
- the cache line size.

We suggest you to fill in Table 1.2 that will help you understand some performance degradation that may occur when increasing the number of threads in future exercises.

Table 1.2: Architecture and memory hierarchy characteristics of any of the boada execution nodes boada-18 to boada-20.

Characteristic	Value
Number of sockets per node	2
Number of cores per socket	16
Number of threads per core	2 important!
Maximum core frequency	3400 MHz
L1-I cache size (per-core)	32 KB
L1-D cache size (per-core)	48 KB
L2 cache size (per-core)	1280 KB
Last-level cache size (per-socket)	24 MB
Main memory size (per socket)	64 GB
Main memory size (per node)	1360+1536 KB
Cache line size (in bytes)	64 B important!

1.3 Execution modes: interactive vs. queued

There are two ways to execute your programs in boada:

Interactively. Using any of the login nodes, from boada-11 to boada-13, all of them limited to a maximum of 1 core (two threads with Hyperthreading) for parallel executions.

Via a queuing system. Using one of the execution nodes, from boada-18 to boada-20.

Interactive executions start immediately and do share resources with interactive programs run by other users logged in to the system, thereby not ensuring representative timing results. In addition to this, the login nodes have a different architecture and memory hierarchy than the execution nodes. The corresponding *interactive* scripts are executed with the command (where ??? refers to a name):

```
./run-??? .sh
```

You must use the second option for running programs that require several processors in a node, ensuring that your job is executed in isolation and therefore reporting reliable performance results. These executions do not overload the interactive/login nodes. Note also that *queued* scripts start

as soon as an execution node is available. The corresponding *execution* scripts must be submitted as a SLURM job:

```
sbatch ./submit-???.sh
```

After submitting your job, you can run the command

```
squeue
```

to ask the system about the job status within the global queue (together with other users' jobs). You can check only the jobs associated with your account by typing the `-u` flag and your user identifier:

```
squeue -u par????
```

Finally, you can cancel a previously submitted job via

```
scancel <jobid>
```

where `<jobid>` refers to the job identifier. Every submitted execution generates two additional files that share the same script name followed by `.e` and `.o` together with the job identifier. Such files contain the messages sent to the standard error and output streams, respectively, which we encourage you to check.

1.4 Serial compilation and execution

The `pi_seq.c` code inside the `lab1/pi` directory computes the value of pi (π) by numerically integrating the equation $1/(1+x^2)$ within the interval $[0, 1]$. The *exact* integral equals $\pi/4$ and hence we use $4/(1+x^2)$ as depicted in Fig. 1.1. The integral is numerically *approximated* by the sum of N rectangles, being more accurate as this number increases.

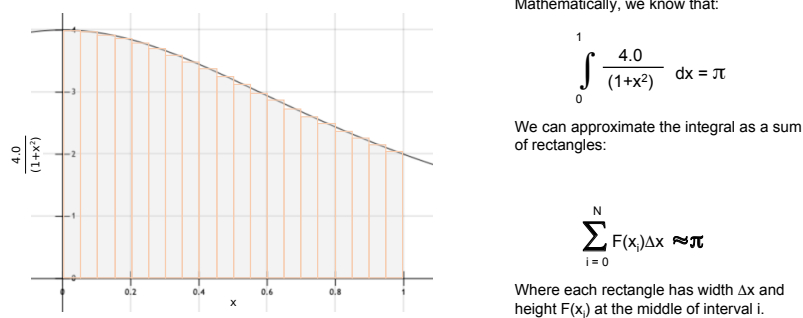


Figure 1.1: Computation of pi (π).

Code 1 shows a simplified version of the `pi_seq.c` code. The variable `num_steps` defines the number of rectangles, whose area is computed at each iteration of the loop.

```
static long int num_steps = 100000;

void main () {
    double x, pi, sum = 0.0, step = 1.0/(double) num_steps;
    for(long int i = 0; i < num_steps; ++i) {
        x = (i + 0.5)*step;
        sum += 4.0/(1.0 + x*x);
    }
    pi = step*sum;
}
```

Code 1: Computation of π .

Figure 1.2 shows the compilation and execution flow for a sequential program. Program

compilation and binary generation is done with a **Makefile** that declares multiple targets with specific rules. This course uses **icx** (Intel's oneAPI C compiler) to generate the binary files.

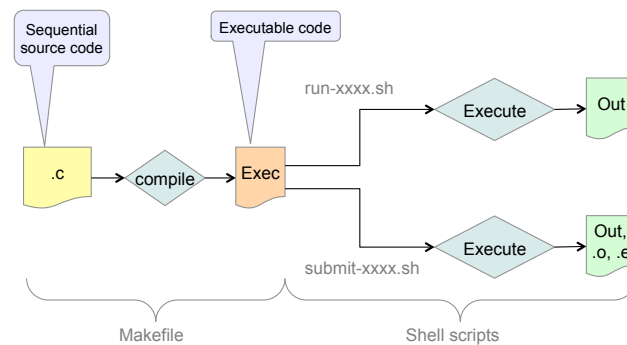


Figure 1.2: Compilation and execution flow for sequential programs.

Now let's compile `pi_seq.c` using the **Makefile** and execute the binary generated interactively and through the queueing system:

1. Open the **Makefile** file and identify the **target** employed to compile the sequential code. Observe how the compiler is invoked.
2. Interactively execute the generated binary file using the command:

```
./run-seq.sh <name> <iterations>
```

with the corresponding executable name and a fixed number of iterations equal to 10^9 . The execution returns the user and system CPU time, the elapsed execution time, and the percentage of CPU used.

3. Look at the source code and identify the function invocations and data structures used to measure the execution time.
4. Submit the execution script to the queueing system via:

```
sbatch submit-seq.sh <name> <iterations>
```

with similar arguments: the binary name and 10^9 iterations. Use the **squeue** command to check your job.

5. Open the generated standard output and error files `time-pi_seq-boada-??`, where `??` refers to the node where the execution took place.

Take a closer look at the `run-seq.sh` and `submit-seq.sh` scripts to understand how the binaries are executed.

1.5 Compilation and execution of OpenMP programs

OpenMP is the standard model for parallel programming using shared-memory. It allows to express parallelism in the C programming language. This section explains how to compile and execute parallel programs with OpenMP.

Figure 1.3 shows the compilation and execution flow for an OpenMP program. The main difference with respect to Fig. 1.2 is that now the **Makefile** includes the appropriate compilation flag that enables OpenMP.

We propose you to carry out the following steps:

1. Open the parallelised code `pi_omp.c` and figure out what the new added lines of code do.
2. Open the **Makefile** and identify the **target** associated with the OpenMP code compilation (the only difference is the `-qopenmp` compilation flag).
3. Take a look at the script to learn how to specify the number of threads to OpenMP.

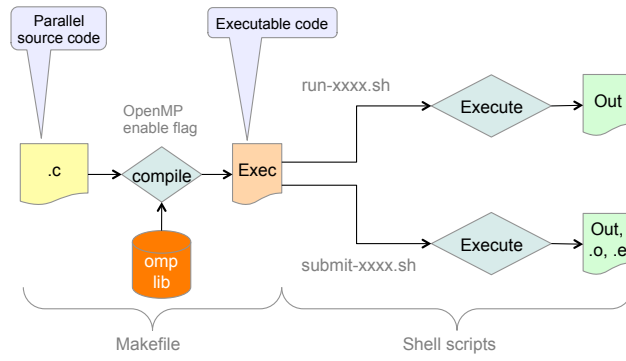


Figure 1.3: Compilation and execution flow for parallel programs using OpenMP.

- Interactively execute the OpenMP code with 1, 2, 4, 8, 16, and 32 threads using 10^9 iterations:

```
./run-omp.sh <name> <iterations> <threads>
```

- What is the `time` command telling you about the user and system CPU time, the elapsed time, and the percentage of CPU used?

- Submit the execution script to the queuing system via:

```
sbatch submit-omp.sh <name> <iterations> <threads>
```

specifying the name of OpenMP binary, 10^9 iterations, and an increasing number of threads: 1, 2, 4, 8, 16, and 32. Look at all the generated files `time-pi_omp-ll-boada-??`, being *ll* the number of threads and ?? the execution node.

We suggest you to fill in Table 1.3 to compare the differences between interactive and queued executions.

Table 1.3: Interactive vs. queued execution times.

Number of threads	Interactive timing information				Queued timing information			
	user	system	elapsed	% CPU	user	system	elapsed	% CPU
1	0.69	0.00	0:00.70	99%	0.69	0.00	0:00.71	98%
2	1.38	0.00	0:00.70	198%	0.69	0.00	0:00.37	188%
4	1.31	0.17	0:00.75	197%	0.70	0.00	0:00.19	353%
8	1.36	0.02	0:00.70	197%	0.69	0.00	0:00.10	646%
16	1.36	0.02	0:00.70	197%	0.69	0.00	0:00.07	1000%
32	1.31	0.13	0:00.73	197%	0.69	0.00	0:00.04	1473%

1.6 Strong vs. weak scalability

This section explores the *scalability* of the parallel version implemented in `pi_seq.c` for an increasing number of threads. It is measured as the ratio between the sequential and the parallel execution times. This ratio is often referred to as *speedup*.

Two scalability scenarios are considered:

Strong. The number of threads increases while the problem size is kept fixed; that is, the parallelisation is employed to reduce the execution time of the program.

Weak. The problem size is proportional to the number of threads; that is, the parallelisation is employed to increase the problem size for which the program is executed.

We provide two scripts that analyse the scalability of `pi_omp.c`. They execute the parallel code using `np_NMIN=1` to `np_NMAX=32` threads. The problem size is set to 10^9 and 10^8 for strong and weak scalability scenarios, respectively. For the latter, the problem size grows proportionally to

the number of threads, as previously explained. Finally, each script generates a plot showing the parallel execution time and the speedup.

We ask you to follow the next steps:

1. Submit the strong scalability script (no arguments are required):

```
sbatch submit-strong-omp.sh
```

and watch the state of your submission via the `squeue` command.

2. Convert the generated Postscript file to PDF via the `ps2pdf` command and visualise it with the `xpdf` command (replace `???` by the corresponding name):

```
ps2pdf pi_omp-???.ps  
xpdf pi_omp-???.pdf
```

and observe how the execution time and speedup varies with the number of threads.

3. Set `np_NMAX=64` within `submit-strong-omp.sh` and repeat the previous steps. Can you explain the behaviour observed in the scalability plot?
4. Submit the weak scalability script (no arguments required):

```
sbatch submit-weak-omp.sh
```

and observe how the parallel efficiency varies with the number of threads.

Remember to take screenshots by pressing the dedicated `Prtsc` keyboard button.

1.7 Discussion at the laboratory

These questions are to be discussed during or at the end of the laboratory with your professor:

- Do you observe any major differences between *interactive* and *queued* executions in Table 1.3?
- Look at the *weak* and *strong* scalability plots generated and reason about the obtained performances.

Appendix A

Accessing the boada cluster

You can access boada from your laptop, booted with Linux, macOS, or Windows, if a secure shell client is installed. To access the boada server from a computer outside the UPC you will need to use UPC's VPN connection.

On Windows you need both *putty* for secure shell:

<https://www.chiark.greenend.org.uk/~sgtatham/putty/latest.html>

and *xming* for X11:

<https://wiki.centos.org/HowTos/Xming>

alternatively, you can use *MobaXterm* which integrates both secure shell and X11:

<https://mobaxterm.mobatek.net/download.html>

On macOS you need *XQuartz* and the option `-Y`:

```
ssh -Y par????@boada.ac.upc.edu
```

where `????` refers to your user number.